



the product to market

- Creates test plans
- Executes test plans effectively

Let's look at examples of how improper test plans can negatively impact product time to market. All of this work happens while closely collaborating with cross-functional teams. Hardware design engineers interact with mechanical teams for PCB dimensions and connector placement, software teams for firmware readiness, sourcing teams for component availability and cost, business development teams to meet cost constraints, and testing and product validation teams. In practice, hardware engineers tend to work with more teams than most other roles.

In addition to all of this, one of the most important responsibilities of a hardware engineer is reading datasheets. This is critical and unavoidable. There are no shortcuts here. Without thoroughly understanding datasheets, nothing else can move forward.

The first time, the right approach

Hardware engineers do not have an undo button. Unlike firmware, where code can be changed and retried, hardware mistakes remain until they are physically fixed. If something is wrong, it cannot be reversed instantly. Even identifying the problem takes time. Issues are rarely obvious. Engineers must probe, debug, remove components, and solder them back. This process is slow, manual, and uncertain. Fixing one issue does not guarantee that others are not hiding. Every problem must be found and resolved before moving forward.

As a result, hardware development cycles stretch quickly. Once an issue is discovered, the design must be updated, components re-sourced, PCBs fabricated, assembled, and tested again. The cycle effectively restarts. This drives up development time, delays time to market,

impacts the business, causes financial loss, and increases pressure on management.

The key requirements to avoid these challenges are:

Collect requirements from all stakeholders. Hardware engineers work with the maximum number of cross-functional teams. Requirements must be gathered from mechanical, hardware, software, testing, and validation teams. Inputs from all these teams must be incorporated into the design.

Understand requirements to the finest details. It is not enough to broadly define a system and select components. Every requirement must be deeply understood. For example, wake-up over CAN is a common requirement today. It may sound simple, but it is not achieved by just connecting a CAN IC to a microcontroller. Each microcontroller behaves differently, and wake-up functionality may only be available on specific pins. Many engineers miss this detail and later discover that the selected CAN RX pin does not support wake-up from low power mode, forcing a redesign and another PCB iteration. Similar fine details exist in power management, under-voltage lockout, and other system-level features, and these must be thoroughly studied and understood.

Sign-off requirements from all stakeholders. After analysing requirements, discussions must happen if there are challenges. Final requirements must be formally signed off by all teams. This is critical. Without a sign-off, teams may later claim that certain requirements were communicated but not implemented. Signed off requirements avoid conflicts, save time, and ensure accountability. Once signed off, there is no ambiguity.

Freeze requirements and stop accepting changes. At some point, requirement changes must stop. Accepting new requirements every day disrupts the design process.

After sign-off, only critical changes should be considered, and their impact on timelines and design effort must be clearly communicated. Changes should be accepted only if all stakeholders agree to the consequences. Freezing requirements are essential to move forward and complete the design efficiently.

How to create a system architecture

Once all requirements are gathered, the next step is to create the system architecture.

Hardware design architecture.

The system architecture must be detailed and must capture every functionality defined in the requirements. Using simple elements such as rectangles, lines, and text, the objective is to translate requirements into a clear and comprehensive architectural representation.

What is often mistaken for architecture is actually just a block diagram. A few blocks connected by lines do not represent a true architecture because they lack detail. A proper architecture includes fine-level details and clearly reflects all system requirements, interfaces, and interactions.

A detailed architecture diagram is extremely useful because requirements are usually scattered across multiple documents, often Excel sheets stored in different locations. In contrast, the architecture diagram can be integrated directly into the CAD tool being used, such as OrCAD or Altium, and can serve as the front page of the design documentation. This allows designers to quickly see what has been implemented and what is still pending. By carefully translating every requirement from various documents into the architecture diagram, a single visual reference is created, improving clarity and ensuring nothing is missed.

Time planning. Once the requirements are frozen, that is where the timeline begins. I break the architecture into smaller functional blocks